

Analyse des Modèles PyTorch et Keras

1. Analyse des Données du Fichier CSV

Voici l'analyse détaillée sous forme de tableaux comparatifs pour les modèles de classification et de régression.

Tableau Comparatif : Classification

Modèle	Fonction	Optimiseur	Couches	Test Accuracy	Train Acc. (Dernière Époque)	Val Acc. (Dernière Époque)	Écart Train-Val	Test Loss
2L_300-100	ReLU	Adam	2	97.73%	99.47%	97.73%	+1.74%	0.104
3L_392-196-98	ReLU	Adam	3	97.98%	99.37%	97.98%	+1.39%	0.088
4L_512-256-128-64	ReLU	Adam	4	98.10%	99.29%	98.10%	+1.19%	0.082
2L_300-100	ReLU	RMSprop	2	97.85%	99.55%	97.85%	+1.70%	0.155
3L_392-196-98	ReLU	RMSprop	3	97.73%	99.43%	97.73%	+1.70%	0.180
4L_512-256-128-64	ReLU	RMSprop	4	97.85%	99.07%	97.85%	+1.22%	0.238

Tableau Comparatif : Régression

Modèle	Fonction	Optimiseur	Couches	Test Loss	Test MAE	Train MAE (Dernière Époque)	Val MAE (Dernière Époque)	Écart Train-Val
2L_300-100	ReLU	Adam	2	0.384	0.272	0.175	0.272	-0.097
3L_392-196-98	ReLU	Adam	3	0.396	0.196	0.141	0.196	-0.055
4L_512-256-128-64	ReLU	Adam	4	0.359	0.170	0.125	0.170	-0.045

2. Comparaison entre Modèles de Régression et Classification

Tableau Comparatif Global : Classification vs Régression

Métrique	Classification (Moyenne)	Régression (Moyenne)	Écart	Analyse
----- ----- ----- ----- -----				
Test Accuracy / MAE	97.76%	0.210	+76.55%	La classification est plus précise.
Test Loss	0.121	0.376	-0.255	La régression a une perte 3x plus élevée.
Overfitting (Train-Val)	+1.65%	-0.065	+1.72%	Léger overfitting en classification.

Synthèse Finale

- Classification : ReLU + Adam (4 couches) est le meilleur compromis (98.10% de précision, overfitting de +1.19%).
- Régression : Sigmoid + Adam (4 couches) est le meilleur (MAE = 0.130, overfitting minimal).

Classement des Meilleurs Modèles

Classification (Top 3) :

1. ReLU + Adam (4L) : 98.10% de précision
2. ReLU + Adam (3L) : 97.98% de précision
3. Sigmoid + Adam (2L) : 98.01% de précision

Régression (Top 3) :

1. Sigmoid + Adam (4L) : MAE = 0.130
2. ReLU + RMSprop (4L) : MAE = 0.158
3. ReLU + Adam (4L) : MAE = 0.170

3. Analyse de ton Modèle PyTorch

Points Positifs

- Précision cohérente : 97.5% à la 2ème époque est dans la fourchette des modèles Keras (97.4%-98.1%).
- Détection de l'overfitting : Un écart de ~1.5% est légèrement inférieur à la moyenne observée dans le CSV (+1.65%).
- Rapidité de convergence : Atteindre 97.5% dès la 2ème époque suggère que les données sont bien séparables.

Pourquoi l'overfitting commence-t-il dès la 2ème époque ?

Causes possibles et solutions :

1. Modèle trop complexe pour la taille des données

-> Réduire le nombre de couches/neurones.

2. Taille du dataset trop petite

-> Vérifier la taille de train_loader vs val_loader.

3. Pas de régularisation

-> Ajouter Dropout(0.2) ou weight_decay dans l'optimiseur.

4. Learning Rate trop élevé

-> Essayer lr=0.0001 au lieu de lr=0.001.

5. Batch Size trop petit

-> Augmenter batch_size (ex: 64 ou 128).

6. Données mal mélangées

-> Utiliser shuffle=True dans DataLoader.

Comparaison avec les modèles Keras

Critère | Ton Modèle PyTorch | Moyenne Keras | Écart

-----|-----|-----|-----

Précision finale | ~97.5% | 97.76% | -0.26%

Overfitting | +1.5% | +1.65% | Meilleur

Convergence | 2 époques | 6-10 époques | Plus rapide

Recommandations

1. Vérifie la taille de ton dataset (idéalement > 10k échantillons).
2. Ajoute du Dropout et relance l'entraînement.
3. Teste avec Early Stopping pour éviter de sur-entraîner.
4. Compare avec un modèle plus simple (ex: 2 couches) pour vérifier si la précision reste > 97%.

Exemple de code pour améliorer ton modèle :

```
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.dropout = nn.Dropout(0.2) # Ajoute ça
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x) # Applique le Dropout
        x = self.fc2(x)
        return x

# Early Stopping
scheduler = ReduceLROnPlateau(optimizer, 'min', patience=2)
for epoch in range(20):
    train(...)
    val_loss = validate(...)
    scheduler.step(val_loss)
```